

Appendix: Tooling and Infrastructure

Appendix: Tooling and Infrastructure I

The practical utility of a computational meta-analysis depends on robust tooling at each pipeline stage: assertion extraction, modeling and simulation, knowledge-graph infrastructure, and quality assurance. This appendix surveys the open-source ecosystem of Active Inference (AIF) and Free Energy Principle (FEP) implementations as of early 2026, documents the engineering trade-offs behind our knowledge-graph backend, and lists the multi-level quality gates enforced by the pipeline.

LLM-Based Assertion Extraction I

Extracting structured assertions from unstructured text is the most labor-intensive component of knowledge-graph construction. Manual annotation produces high-quality results but does not scale to corpora of thousands of papers—a constraint demonstrated by Knight et al. [knight2022fep], whose systematic analysis of FEP and Active Inference publications required manual coding of structural, visual, and mathematical features for hundreds of annotated papers. We implement a hybrid approach: an LLM performs initial extraction and human review provides validation pathways.

LLM-Based Assertion Extraction II

Our extraction pipeline deploys a locally hosted LLM through Ollama [ollama2024]. Each paper's abstract is assessed against the eight hypothesis definitions in a structured prompt requesting a JSON array of assessments. Unlike keyword matching, which detects only topical terms, the LLM evaluates the *semantic relationship* between a paper's claims and each hypothesis. Papers critiquing the FEP correctly receive “contradicts” assessments for FEP Universality (H1), while methodology tutorials receive “neutral” assessments reflecting their pedagogical character. Detailed prompt engineering, schemas, and failure modes are documented in the extraction pipeline section.

Software Ecosystem I

The Active Inference community has developed a rapidly growing ecosystem of open-source tools spanning multiple programming languages, inference paradigms, and application domains. This section provides a comprehensive survey of publicly available implementations as of early 2026, organized by functional category. We emphasize tools with accessible source code: open-source availability is a prerequisite for reproducibility and community-driven validation.

Software Ecosystem II

General-Purpose Frameworks

Six general-purpose frameworks dominate the landscape, collectively covering discrete, continuous, and real-time inference: **pymdp**. The pymdp library [heins2022pymdp] provides a Python implementation of active inference for discrete state-space POMDPs, supporting message passing on factor graphs, policy inference via expected free energy, and hierarchical generative models. It has become the standard entry point for algorithm development and the most widely forked AIF repository.

SPM. The SPM package (Wellcome Centre for Human Neuroimaging) includes MATLAB implementations of Dynamic Causal Modeling and variational Bayesian inference under the FEP. It remains the reference implementation for neuroimaging applications and houses the original Friston-group POMDP scripts.

RxInfer.jl. RxInfer is a Julia package for reactive message-passing-based Bayesian inference, supporting real-time and streaming inference suitable for robotics and online learning. Version 4.0.0 (early 2025) [rxinfer2025] introduced projected constraints and adaptive inference optimized for dynamic data

Knowledge Graph Infrastructure I

Our knowledge graph uses an RDF-compatible schema deployable on standard semantic-web infrastructure. The nanopublication model [groth2010anatomy, kuhn2016decentralized] provides a principled atomic unit of scientific evidence: each nanopublication packages a single assertion (e.g., “Paper X supports Hypothesis Y”) with explicit provenance and publication metadata in four named RDF graphs (Head, Assertion, Provenance, Publication Info). This structure satisfies the FAIR data principles by design: nanopublications are **F**indable via URI-based identification, **A**ccessible through standard RDF protocols, **I**nteroperable via W3C-standard TriG serialization, and **R**eusable with explicit provenance and CC0 licensing. The full RDF schema and a TriG serialization example are presented in the methodology and Appendix~??.

The engineering trade-offs among the three deployment options are straightforward:

Knowledge Graph Infrastructure II

Nanopublication servers provide decentralized, content-addressed storage. The pipeline writes nanopublications in two forms: JSON Lines (for incremental checkpointing and tooling) and RDF/TriG per the nanopublication standard (Assertion, Provenance, Publication Info), suitable for the nanopublication network and FAIR deployment. The recent release of nanopub-js v0.1.0 [kuhn2026nanopubjs]—a JavaScript library enabling browser-based creation, signing, and querying of nanopublications—opens the possibility of community-contributed assertions directly from web interfaces, lowering the barrier to participatory evidence curation. Future integration with Trusty URIs [kuhn2014trusty] would provide cryptographic content verification and persistent identifiers for each nanopublication.

RDF stores (e.g., Apache Jena Fuseki, Blazegraph, Oxigraph) enable SPARQL queries such as “find all papers supporting hypothesis H published after 2020 in the neuroscience domain (C1).” The cost is operational overhead and query latency.

Knowledge Graph Infrastructure III

Property-graph databases (e.g., Neo4j) prioritize traversal performance for path queries and community detection, at the expense of semantic-web compatibility.

While RDF and property graphs excel at structurally organizing assertions, it is crucial to recognize that they inherently compress the rich epistemic context of the original papers (e.g., methodological caveats, sample sizes, scope limitations) into flattened confidence scores—a fundamental limitation of current automated knowledge extraction discussed in the conclusion.

The Active Inference Ontology namespace ensures integration with external ontologies and linked-data resources.

Multi-Level Quality Assurance I

Quality assurance operates at four levels: assertion-level confidence and review, graph-level structural consistency, score-level boundary tests, and pipeline-level continuous-integration coverage.

Assertion-Level Validation

Assertions below a configurable confidence threshold (default 0.6) are flagged for review. The threshold is chosen to balance recall against the prompt-engineering cost of pushing the LLM to over-commit; lowering it inflates noisy neutral assertions, raising it discards weakly supported but legitimate claims. There is no live per-assertion multi-annotator mechanism; instead, an offline stratified sample is checked against a deterministic rule-based reference (see the extraction-agreement study in the main methods section)—a reproducibility floor, not human inter-annotator agreement.

Multi-Level Quality Assurance II

Graph-Level Consistency Checks

Consistency checks verify that all nodes link to valid targets and no orphan nodes exist. Coverage metrics track the proportion of annotated papers, the fraction of references that resolve inside the corpus, and the per-domain assertion density.

Score-Level Unit Testing

Hypothesis scoring is validated through unit tests on synthetic data verifying boundary conditions: all-support fixtures must produce scores at $+1$, all-contradict at -1 , and balanced inputs at 0 . Sensitivity analysis sweeps over confidence thresholds and citation-weighting schemes to confirm that qualitative rankings are stable.

Multi-Level Quality Assurance III

Pipeline-Level Test Coverage

Test-driven development enforces 90% minimum code coverage on project modules and 60% on shared infrastructure, with real data and computation (no mocking). All tests run on every push; failures block merges and releases.

Multi-Level Quality Assurance IV

Quality Thresholds

Table 2: Multi-level quality-assurance thresholds enforced across the pipeline. Each level defines a metric, minimum threshold, and failure action. Pipeline-level thresholds (90% coverage, 100% pass rate) are enforced via CI gates; lower-level checks emit warnings or block release as indicated.

Level	Metric	Threshold	On Failure
Assertion	Confidence c	≥ 0.6	Flag for review
Assertion	Rule-reference κ (offline sample)	Reported, not gated	Report only
Graph	Orphan-node ratio	$= 0$	Reject build
Graph	Corpus coverage	$\geq 80\%$	Warning
Score	Boundary tests (all-support / all-contradict / balanced)	All pass	Block release
Score	Sensitivity-sweep stability	Top- k ranks unchanged	Warning
Pipeline	Project-code coverage	$\geq 90\%$	Block merge
Pipeline	Infrastructure coverage	$\geq 60\%$	Block merge
Pipeline	Test pass rate	100%	Block release

The hypothesis-evidence results, temporal dynamics of evidence accumulation, and assertion analysis are presented in the hypothesis results section.